Tema 2

Manejo de la sintaxis del lenguaje



ÍNDICE

01 Variables

02 Tipos de datos

03 Asignaciones

04 Operadores

05 Comentarios al código

06 Sentencias

07 Decisiones

08 Bucles

09 Pruebas

Variables

- Una variable es un nombre que apunta a un valor.
- Pueden declararse con **const** o con **let**.
  - **const**: para valores inmutables.
  - **let**: para valores mutables.
- El **ámbito** es la zona del código donde una variable existe. En el caso de **let** y **const**, ambas tienen ámbito de bloque, por lo que solo existen dentro de las llaves donde se declaran.

Tipos de datos

- JavaScript tiene **tipado dinámico**: no declaras el tipo de una variable, sino que cada valor tiene el suyo y una misma variable puede guardar primero un número y después un texto.
- Hay 7 tipos **primitivos** (valores simples e inmutables) y el tipo **objeto**, que agrupa todo lo demás (objetos, arrays, funciones, fechas, etc.).

Tipos de datos

Tipo	Ejemplos	typeof	A tener en cuenta
number	42, 3.14, -7, NaN, Infinity	“number”	Enteros y decimales son el mismo tipo (coma flotante de 64 bits
bigint	9007199254740993n	“bigint”	Enteros de cualquier tamaño; no se mezclan con number sin convertir
string	"hola", 'hola', `hola ${nombre}`	“string”	Inmutables: los métodos devuelven textos nuevos
boolean	true, false	“boolean”	El resultado de cualquier comparación
undefined	undefined	“undefined”	Variable sin valor, propiedad inexistente, función sin return
null	null	“object”	«Vacío a propósito». Lo de «object» es un error histórico
symbol	Symbol("id")	“symbol”	Identificadores únicos, sobre todo en librerías
object	{ nombre: "Ana" }, [1, 2], new Date()	“object”	Arrays incluidos: para distinguirlos usa Array.isArray(x)
(function)	function f() {}	“function”	Las funciones son objetos que se pueden llamar

Asignaciones

➔ El operador = se utiliza para asignar valores a las variables.

```
let x = 4
let y = 5
```

Operador	Descripción
=	Asigna a la variable de la parte izquierda el valor de la parte derecha.

Operadores aritméticos

➔ Se utilizan para llevar a cabo cálculos aritméticos.

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo

Operadores de comparación

➔ Se usan para comparar dos valores entre sí.

➔ El valor de retorno siempre es un booleano: *true* o *false*.

Operador	Descripción
===	Compara dos elementos, incluyendo su tipo interno. Si son de distinto tipo, no realiza conversión y devuelve false ya que siempre los considera diferentes. Uso recomendado.
!==	Compara dos elementos, incluyendo su tipo interno. Si son de distinto tipo, no realiza conversión y devuelve true ya que siempre los considera diferentes. Uso recomendado.
==	Devuelve el valor true cuando los dos operandos son iguales. Si los elementos son de distintos tipos, realiza una conversión. No está recomendado su uso.
!=	Devuelve el valor true cuando los dos operandos son distintos. Si los elementos son de distintos tipos, realiza una conversión. No está recomendado su uso.

Operadores de comparación

- Se usan para comparar dos valores entre sí.
- El valor de retorno siempre es un booleano: *true* o *false*.

Operador	Descripción
>	Devuelve el valor true cuando el operando de la izquierda es mayor que el de la derecha.
<	Devuelve el valor true cuando el operando de la izquerda es menor que el de la derecha.
>=	Devuelve el valor true cuando el operando de la izquierda es mayor o igual que el de la derecha.
<=	Devuelve el valor true cuando el operando de la izquierda es menor o igual que el de la derecha.

Operadores de incremento

➔ Se utilizan para aumentar o disminuir en 1 el valor de una variable.

Operador	Descripción
++	Incrementa en 1 la variable.
- -	Disminuye en 1 la variable.

➔ **IMPORTANTE:** En caso de asignación con incremento, es importante el orden de la variable incrementada. No es lo mismo **y = ++x** que **y = x++**.

Operadores aritméticos abreviados

➔ Se utilizan para llevar a cabo cálculos aritméticos de manera abreviada.

Operador	Descripción
+=	Suma los operandos izquierdo y derecho y asigna el resultado al operando izquierdo.
-=	Resta el operando derecho del operando izquierdo y asigna el resultado al operando izquierdo.
*=	Multiplica ambos operandos y asigna el resultado al operando izquierdo.
/=	Divide ambos operandos y asigna el resultado al operando izquierdo.

Operadores booleanos

➔ Se utilizan para procesar valores booleanos.

➔ Devuelven valores booleanos: *true* o *false*.

Operador	**Descripción**
&&	AND: Devuelve *true* cuando ambos operandos son verdaderos.
||	OR: Devuelve *true* cuando alguno de los operandos es verdadero o ambos lo son.
!	NOT: Devuelve *false* si el valor es *true* y *true* cuando el valor es *false*.

Comentarios al código

- Sirven para documentar brevemente el código.
- Pueden ser de dos tipos:
  - De una línea -> //
  - De varias líneas -> /* */

Sentencias

- ➔ Son instrucciones que realizan determinadas acciones.
- ➔ Pueden declarar una variable, ser llamadas a funciones, etc.
- ➔ Una sentencia termina con un punto y coma. Aunque se inserta por defecto con ASI (*Automatic Semicolon Insertion*), añadirlo explícitamente puede evitar posibles errores.

Decisiones: if/else

- La instrucción if/else permite la ejecución de un bloque u otro de instrucciones en función de una condición.
- En el ejemplo, en lugar de usar **else if** también podrían haberse anidado las condiciones.
- **Sintaxis:**

```
if (condition) {
    // Instrucciones si "condicion" se cumple
} else if (otherCondition) {
    // Instrucciones si "condition" no se ha cumplido, pero "otherCondition" sí
} else {
    // Instrucciones si ninguna condición se ha complido
}
```

Decisiones: switch

- Se define cada caso con un **break**.
- Sin **break** se continúa en el siguiente caso, por lo que comparten instrucciones.
- **Sintaxis:**

```
switch(variable) {
    case "x":
        // Instrucciones del caso particular
    break;
}
```

Bucle for

➔ Cuando el programa llega a un bucle **for**:

- Se inicializa el índice. Esto solo se hace una vez.
- Se analiza la condición de prueba y, en caso de cumplirse, se ejecutan las instrucciones del bucle.
- Cuando termina la ejecución de las instrucciones del bucle, se modifica el índice, se vuelve a la cabecera del bucle en ejecución y se lleva a cabo la condición de prueba de nuevo.
- Si la condición se cumple, se ejecutan las instrucciones y, si no, continua con las líneas de código posteriores al bucle.

Bucle for

➔ **Sintaxis:**

```
for (indexInicialization; condition; indexModifier) {
    // Instrucciones del bucle
}
```

Bucle while

- Sirve para la ejecución de instrucciones mientras se cumpla una condición determinada:
  - Si la condición nunca se cumple, no se ejecuta ninguna instrucción.
  - Si la condición se cumple siempre, será un bucle infinito.
    - Es importante que la condición deje de cumplirse en cierto punto para evitarlos.
- **Sintaxis:**

```
while (condition) {
    // Instrucciones del bucle
}
```

Bucle do-while

➔ A diferencia del bucle **while**, el bucle **do-while** no comprueba la condición hasta el final, por lo que el cuerpo de instrucciones se recorrerá al menos una vez.

- Si la condición no se cumple, sale del bucle.
- Si se cumple, vuelven a ejecutarse las instrucciones.

➔ **Sintaxis:**

```
do {
    // Instrucciones del bucle
} while(condition);
```

Bucles: instrucciones *break* y *continue*

➔ Dentro de un bucle, podemos usar las instrucciones **break** y **continue** de la siguiente manera:

- **break:** Interrumpe la ejecución del bucle aunque no se haya ejecutado por completo. El programa sigue a continuación del bucle.
- **continue:** Retorna a la cabecera del bucle, por lo que se ejecuta de nuevo la condición (y se incrementa el índice si es un bucle **for**).

Pruebas

➔ **Pruebas:**

- **DevTools:** usando la pestaña **Consola** en el navegador (Google Chrome, Firefox Developer Edition, etc.).
- **Breakpoints:** usando puntos de ruptura para depurar el código.

➔ **Documentación:**

- Comentarios en el código con los formatos vistos anteriormente y el formato JSDoc.